CS2 Stats & Leaderboard: Setup Guide

For Trevor · Unraid setup · adds Leetify-style stats, a leaderboard website and match scorecards to the existing match-bot

What this adds

Everything runs next to the CS2 server and uses the **same Discord bot** you already have. Nothing on the CS2 server or in AMP needs to change: MatchZy is already recording a demo of every map.

```
MatchZy records a demo -->  .../csgo/MatchZy/<date>_<matchid>_<map>_team_x_vs_team_y.dem
                                |  (read-only; checked every 20s, read once the file stops growing)
                                v
  matchbot-stats      - parses the demo, stores stats in SQLite (appdata/matchbot-stats)
                      - serves the leaderboard website on port 8080
                                |  (http://<Unraid IP>:8080)
                                v
  match-bot           - /leaderboard, /stats, /match, /matches, /link ...
                      - posts a scorecard to the stats channel when a match finishes
```

Only real, finished competitive maps count. Warmup, knife rounds, retakes, wingman, deathmatch, abandoned games and AFK players are filtered out automatically.

Before you start

- Unraid with the Docker tab, which already runs the match-bot container and AMP.
- A Discord channel for match scorecards (for example **#cs2-stats**).
- MatchZy demo recording left on. It's on by default, and the server log shows `[StartDemoRecoding]` lines.
- From Community Applications (Apps tab): **User Scripts**, only if you want automatic demo cleanup later.

Step 1: Get Sam's changes into your repo

Sam's changes are on a branch called `cs2-stats`. Merge that branch into `main` on your `match-bot` repo on GitHub (open a pull request from it and click **Merge**). The repo's existing GitHub Action then builds two images (watch it under the **Actions** tab, about 3 minutes):

Image	What it is
`ghcr.io/trev9575/match-bot:latest`	The Discord bot you already run, now with the stats commands.
`ghcr.io/trev9575/match-bot-stats:latest`	New: the stats service and leaderboard website.

GitHub makes new packages **private**. If your existing match-bot package is public, do the same for the new one: GitHub › your profile › **Packages** › **match-bot-stats** › Package settings › Change visibility › Public. Otherwise Unraid can't pull it (or add your GitHub login the same way you did for match-bot).

Step 2: Find MatchZy's demo folder

The stats container reads demos straight from AMP's files. In the Unraid **Docker** tab, click the AMP container › **Edit**, and look at the path mapped to AMP's data folder (for example `/mnt/user/appdata/amp`). The demos are inside it at:

```
<AMP data folder>/instances/<CS2 instance name>/counter-strike2/730/game/csgo/MatchZy
```

Check it with Unraid's file browser (**Shares** › appdata, or the folder icon in the header): it should contain `.dem` files named like `2026-11-04_20-20-59_3_de_wrecked_team_..._vs_team_....dem`. Copy that full path for the next step.

Step 3: Add the stats container

Docker tab › **Add Container**. Turn on **Advanced View** (top right), then fill in:

Field	Value
Name	`matchbot-stats`
Repository	`ghcr.io/trev9575/match-bot-stats:latest`
Network Type	`Bridge`
WebUI	`http://[IP]:[PORT:8080]/`
Extra Parameters	`--cpus=1 --memory=1g --log-opt max-size=5m --log-opt max-file=2`

Then use **Add another Path, Port, Variable** for each of these:

Type	Name	Container	Host / value
Port	Website	8080	8080 (or any free port)
Path	Demos	/demos	the MatchZy folder from Step 2 · **Access Mode: Read Only**
Path	Database	/data	/mnt/user/appdata/matchbot-stats
Variable	API key	STATS_API_KEY	any long random string, e.g. from a password generator. Needed again in Step 4.

Click **Apply**. Once it's running, click its icon › **WebUI**: the leaderboard should open with the four games Sam already imported. On first start it also reads every demo already in the MatchZy folder, quietly (no Discord spam), and then checks for new ones every 20 seconds. Click the icon › **Logs** to watch: lines like `imported ..._de_ancient_....dem as match #5` mean it's working.

Step 4: Connect the bot to it

Docker tab › your existing match-bot container › **Edit**. Add these variables (**Add another Path, Port, Variable** › Config Type: Variable), then click **Apply**. Unraid pulls the new image and restarts the bot, and the new slash commands register automatically.

Key	Value
`STATS_API_URL`	http://<Unraid IP>:8080 (e.g. http://192.168.1.50:8080)
`STATS_API_KEY`	exactly the same string as in Step 3

Key	Value
`STATS_CHANNEL_ID`	the scorecard channel: in Discord turn on Developer Mode (Settings › Advanced), right-click the channel › Copy Channel ID
`STATS_PUBLIC_URL`	the address friends open (Step 5). Used for the link buttons in Discord; leave it empty until the site is reachable.
`STEAM_API_KEY`	optional: lets /link accept custom steamcommunity.com/id/name links (steamcommunity.com/dev/apikey)

If the repository on the match-bot container isn't `ghcr.io/trev9575/match-bot:latest` (for example it was built locally), switch it to that so it picks up the new version. Use **Check for Updates** on the Docker tab if it doesn't pull right away.

Step 5: Make the website reachable for friends

Pick one, then put the address into `STATS_PUBLIC_URL` on the match-bot container (Step 4).

Option A: Port forward (simplest)

- On the router, forward TCP port **8080** to the Unraid server's IP.
- `STATS_PUBLIC_URL` = `http://<your public IP>:8080` (look it up at ifconfig.me).
- People on the same home network as the server may need the LAN address instead, the same quirk as the CS2 connect button.

Option B: Cloudflare Tunnel (HTTPS, no open ports)

Install the **cloudflared** app from Community Applications, create a tunnel in the Cloudflare Zero Trust dashboard, and point a public hostname (e.g. `stats.yourdomain.com`) at `http://<Unraid IP>:8080`. Free, gives https://, and doesn't expose your home IP.

Option C: Nginx Proxy Manager or SWAG

If you already run one for other services, add a proxy host for a subdomain pointing at `<Unraid IP>:8080`.

The website is read-only. Linking accounts, deleting matches and the scorecard queue all need STATS_API_KEY, which only the bot has.

Step 6: Discord permissions

The bot needs **View Channel**, **Send Messages** and **Embed Links** in the stats channel. No new privileged intents are needed. Then tell everyone to run **/link** once with their Steam profile URL. That connects their Discord account to their Steam ID, so **/stats** works with no arguments and the MVP gets tagged in each scorecard.

Step 7 (optional): Let the bot send teams to the server

After teams are picked (by draft or by **Randomize Teams**), the bot can load them into MatchZy, so everyone lands on the right team and the knife round decides sides. MatchZy downloads the teams from the stats website, so `STATS_PUBLIC_URL` (Step 5) has to be set. Two things to know first:

- Everyone in the lobby must have run **/link**. Once teams are loaded, MatchZy kicks anyone who isn't on one until the match ends. The bot checks this and names anyone missing before it sends anything.
- Workshop maps only load by their workshop **ID number** (the number in the map's Steam Workshop link). Standard maps load by name.

On the CS2 server: set an RCON password. In AMP that's the CS2 instance's server settings (or `rcon_password "something-long"` in server.cfg). If the bot later says it can't reach RCON, add `-usercon` to the server's launch parameters in AMP and restart the instance.

On the match-bot container (Edit › add variables, then Apply):

Key	Value
`CS2_RCON_PASSWORD`	the RCON password above
`CS2_RCON_HOST`	the Unraid LAN IP, e.g. 192.168.0.204 (not the public address)
`CS2_RCON_PORT`	27015, the game port (only if it's different)
`MAP_POOL`	maps offered in the menu, comma-separated. Workshop maps as Name=ID. e.g. de_mirage,de_inferno,de_ancient,Wrecked=1234567890

Without RCON it still works: the bot shows a one-line `matchzy_loadmatch_url ...` command to paste into the AMP console instead.

Test it once with a few people before a real match night: pick **Keep the current map**, and check everyone lands on the right team. If a workshop map ID doesn't load, use Keep the current map after changing to it in AMP.

New Discord commands

Command	What it does
`/leaderboard [stat] [period] [min_matches]`	Ranked table, 10 players per page with Prev/Next buttons, plus a button to the full website. Rank by Rating 3.0, Elo/rank, K/D, ADR, KAST, swing, headshots, clutches, utility, Leetify-style category scores and more.
`/stats [player] [member]`	A player's profile: rating, category percentiles, duels, utility and recent matches. Defaults to you once you've linked.
`/startmatch (buttons)`	The lobby now has **Randomize Teams (by Elo)**: picks 2 random captains and splits everyone else so both teams' average Elo is as close as possible. Only the person who started the lobby (or a server admin) can click it. After any draft, **Send Teams to Server** loads the teams into MatchZy (Step 7).
`/match [id]`	Full scorecard for the latest match, or a specific one, including each player's Elo change.
`/matches`	The last 10 matches, with links to each scorecard.
`/link <steam> · /unlink`	Connect or disconnect your Discord and Steam accounts.
`/deletematch <id>`	Admins only (Manage Server). Removes a test or bad game from the stats. The demo file is untouched.
`/rescan`	Admins only. Checks the demo folder immediately instead of waiting up to 20 seconds.

After every match, the bot posts the scorecard to the stats channel on its own, about a minute after the map ends: both teams' K-D-A, ADR and rating, plus highlights (MVP, entry king, best clutch, aces, nade merchant, flash god). If the MVP has run /link, they get tagged.

Disk space and resources

Thing	Size	Notes
Stats database	a few KB per match	About 10 MB after 1,000 matches. Only per-player totals and a round-by-round result are stored, not the demos.
Stats image	~400 MB	Python plus the demo parser, in Unraid's docker image.
CPU / RAM	capped at 1 core / 1 GB	Parsing one demo takes a few seconds. Idle otherwise.
Container logs	capped at 10 MB	Set by the Extra Parameters in Step 3.
MatchZy demos	50–100 MB per map	Written by MatchZy, not by this. They pile up over time; see below.

The stats service never deletes demos (it can't: the folder is mounted read-only). Once a demo is imported, its stats live in the database, so old demos can be removed safely. To keep only the last 30 days: Settings › **User Scripts** › Add New Script, paste this (with your MatchZy folder), and set the schedule to **Weekly**:

```
#!/bin/bash
find "/mnt/user/appdata/amp/instances/<instance>/counter-strike2/730/game/csgo/MatchZy" -name '*.dem' -mt
```

> Use the same folder as the Demos path in Step 3. Keep demos you might want to rewatch somewhere else first.

Backups and updates

- **Backup:** everything that matters is `/mnt/user/appdata/matchbot-stats/stats.db`. The Appdata Backup plugin picks it up automatically.
- **Update:** after a change is merged on GitHub, Docker tab › **Check for Updates** › apply the update on both containers. The database upgrades itself.
- **Start over:** stop matchbot-stats, delete its appdata folder, start it again. It re-imports every demo still in the MatchZy folder.

Troubleshooting

Problem	Check
Container won't pull the image	The new package is still private on GitHub (see the note in Step 1), or the Action hasn't finished yet.
Website won't load from outside	Try the WebUI from inside your network first. If that works, it's the port forward or tunnel (Step 5).
No scorecard after a match	matchbot-stats › Logs: look for “imported” or “skipped … (reason)”. Then match-bot › Logs: look for “[match feed]”. Check STATS_CHANNEL_ID, that STATS_API_KEY matches on both containers, and the channel permissions.
A real match was skipped	The log gives the reason (e.g. “only 9 rounds”, “team sizes 3v5”, “unfinished 12:10”). For MR8 short matches set WIN_ROUNDS=9. For smaller games lower MIN_TEAM_SIZE.
“The stats service isn't reachable” in Discord	Is matchbot-stats running? Is STATS_API_URL the Unraid IP and the port from Step 3 (not localhost)?
Logs say the demo folder is empty or missing	The Demos path in Step 3 is wrong. Compare it with the AMP container's path mapping and check it in the file browser.
Commands don't show up	match-bot › Logs should say “Commands registered!” after the update. Discord can take a minute; restarting the Discord app helps.

How the numbers work

- **Rating 3.0:** an approximation of HLTV's Rating 3.0 (Swing 33%, Kills 25%, Survival 14%, KAST 12%, Damage 11%, Multi-kills 5%), eco-adjusted. **1.00 = the average player in our games**, not pros.
- **Round swing:** how much a player's kills and deaths changed their team's chance of winning each round.
- **Rank & Elo:** works like CS:GO matchmaking, which Valve built on Glicko-2. Only wins and losses move it; it moves more while a rating is uncertain, and uncertainty grows again after time away. Everyone starts at 1500 (Gold Nova III), ranks show after 5 placement matches, and a rank is hidden after 30 days without playing. The 18 rank bands are our own spacing (Valve never published theirs). Settings on the stats container: PLACEMENT_MATCHES, RANK_EXPIRE_DAYS.
- **Rank badges:** the site draws simple colored badges. To use your own images instead, put files named `1.png` to `18.png` (Silver I to Global Elite) in `/mnt/user/appdata/matchbot-stats/ranks/`.
- **Category scores** (Firepower, Entrying, Trading, Clutching, Utility): 0–100 percentiles within the group, like Leetify's profile.
- Kills and deaths were checked against the server's own scoreboard for every player on four maps, and matched.

Appendix: running it with docker compose instead

If you'd rather use the terminal (or the Compose Manager plugin), the repo has a `docker-compose.yaml` that runs both containers. Copy `.env.example` to `.env`, fill in the stats section (including `MATCHZY_DEMO_DIR`), then run `docker compose up -d --build`. With compose, the bot reaches the stats service by name, so STATS_API_URL is set for you.